# Bubble Sort com MPI no Modelo Divisão e Conquista

Trabalho 3 — Programação Paralela — Grupo 20 (conta LAD `cp32020`)

Gabriel Coelho  Gabriel Cherubini

## 1 Problema

Ordenar $N$ inteiros com bubble sort (BS) em MPI por *divisão e conquista*: numa árvore binária de processos, cada nó *conquista* (ordena, se o pedaço tem até $\delta$ elementos) ou *divide* (manda as metades aos filhos e intercala o que volta). Entrada: pior caso $(N, \ldots, 1)$, $N \in \{10^4, 10^5, 10^6\}$; a raiz confere se o resultado é $1, \ldots, N$.

Como o BS custa $\Theta(n^2)$, dividir reduz o *trabalho total*: com $L$ folhas de $N/L$ elementos, a soma cai para $N^2/L$. Por isso o speed-up pode passar de $p$ (superlinear) sem erro de medição.

## 2 Implementação

Um único fonte (`bs.c`) gera o sequencial (`gcc`) e o paralelo (`mpicc -DPARALELO`), com duas árvores escolhidas por `-v`:

**Árvore clássica** (pseudocódigo do enunciado). Numeração de heap: filhos de $r$ são $2r{+}1$ e $2r{+}2$, pai é $\lfloor (r{-}1)/2 \rfloor$. O filho descobre o tamanho com `MPI_Probe` + `MPI_Get_count`; envios e recepções são bloqueantes. Os nós internos só dividem e intercalam; apenas as folhas ordenam. Com $p = 2^k{-}1$ processos há $L = (p{+}1)/2$ folhas, e $\delta = \lceil N/L \rceil$ faz a árvore usar exatamente os $p$ processos (com outro $\delta$ o programa aborta, em vez de deixar ranks esperando para sempre).

**Árvore do artigo** (Radenski, PDPTA'11). Cada processo fica com a metade inicial e envia a final ao ajudante $r + 2^{nível}$, até não haver mais ajudante, e então ordena o que sobrou: todos ordenam, inclusive os internos. O original usa mergesort; usamos o *mesmo* BS, intercalação e entrada da clássica, para comparar só o balanceamento. Com $p = 2^k$ todos os pedaços são iguais; com $p = 2^k{-}1$ um rank fica com o dobro (4× o trabalho, pelo custo quadrático).

**Envio bloqueante, não `MPI_Isend`.** O artigo envia a metade com `MPI_Isend`, mas o Open MPI só transfere mensagens grandes quando o remetente volta a chamar o MPI, e o BS não chama: os dois ordenavam em série (0,202 s no ensaio com $p = 2$, contra ≈0,10 s esperados).

**Correções no material dado.** O pseudocódigo perdia um elemento com tamanho ímpar (a direita agora leva `tam - tam/2`); a intercalação lia `vetor[tam]` e não liberava o vetor auxiliar.

## 3 Metodologia

Atlântica, 2 nós exclusivos (2× Xeon E5520 2,27 GHz: 8 núcleos físicos / 16 hardware threads por nó), gcc 9.4.0, Open MPI 4.1.1, `-O2`. Ranks em *round-robin* entre os nós. Tempo: na raiz, do `MPI_Barrier` (depois de gerar o vetor) ao fim da última intercalação; no sequencial, só a chamada ao BS. Cada ponto é a mediana de 3 execuções; só o sequencial de $10^6$ (16,6 min) roda uma vez, e $p = 1$ só é medido para $10^4$ e $10^5$. $S = T_{seq}/T_p$ e $E = S/p$ contando *todos* os processos; acima de 32 processos (mais que as hardware threads) usamos $S/32$. A carga de cada rank (Tab. 4) é coletada por `MPI_Gather` fora da medição.

## 4 Speed-up e eficiência

O sequencial leva 0,098 s, 10,0 s e 993,6 s (≈ 100× a cada 10× em $N$). Na árvore clássica com $L$ folhas o ideal é $S = L^2$, e a medida chega perto até 15 processos: com $10^6$, $S = 3{,}98$; 15,5; 60,3 para $p = 3, 7, 15$ ($L^2 = 4, 16, 64$), e $E$ sobe de 1,33 para 4,02 (Tab. 1, Fig. 1). Com $p = 31$ ($L^2 = 256$) o ganho é só 146 ($E = 4{,}70$). As 16 folhas disputam os 16 núcleos físicos com os 15 nós internos, que esperam em *polling* ativo no `Recv`. Por isso cada folha ordena em 5,7 s em média, contra 3,9 s se tivesse um núcleo só para ela. Com $N = 10^4$ o BS some diante do custo fixo (cerca de 10 ms de lançamento e comunicação), e $E$ cai de 1,17 ($p = 3$) para 0,32 ($p = 31$).

## 5 Balanceamento nos níveis da árvore

Na clássica com $p = 15$ e $10^6$, os 7 nós internos ficam 0% ocupados: esperam os filhos por 17–18 s e intercalam em 5 ms, porque a intercalação é $O(n)$ e o BS é $O(n^2)$. As 8 folhas ocupam 92% do tempo, com desbalanceamento (máx./média do BS) de 1,08 (Tab. 4, Fig. 2). Na árvore do artigo com $p = 16$, todos ordenam 62.500 elementos (4,1 s) e ficam 83% ocupados. Com $p = 15$, o rank 7 não tem ajudante ($7 + 8 = 15$) e fica com 125.000: 16,3 s, enquanto os demais terminam em 4,1 s e passam 75% do tempo esperando por ele (desbalanceamento 3,3). Na Fig. 2, *envio* inclui esperar o pai terminar de ordenar para receber o pedaço de volta.

## 6 Ganho com HT

De 15 para 31 processos com $10^6$, o tempo cai 2,4× na clássica e 3,4× na do artigo (Tab. 3, Fig. 4). A maior parte desse ganho vem da árvore, não do HT: dobrar as folhas corta o trabalho total pela metade. Para isolar o HT, comparamos os mesmos 31 processos nas 16 threads de cada nó e presos só aos 8 núcleos físicos (`taskset`). O HT dá 1,18× na clássica e 1,05× na do artigo, onde o rank com o pedaço duplo termina sozinho. Com $10^4$ e $10^5$ a diferença fica no ruído (<20 ms).

## 7 Mecanismos de balanceamento

*Pedaços iguais.* Na árvore do artigo todos ordenam, e com $p = 2^k$ o ideal é $S = p^2$: com $10^6$, $S = 62$ e 240 para $p = 8$ e 16 (64 e 256) e 598 para $p = 32$, onde o HT pesa (Tab. 2). Com $p = 2^k{-}1$ o caminho crítico é um pedaço de $N/2^{k-1}$, como na folha clássica: $p = 15$ dá o mesmo tempo nas duas árvores (16,3 × 16,5 s), e $p = 7$ o mesmo que $p = 4$. Com $p = 31$ o artigo vence (4,8 × 6,8 s) porque o rank mais carregado segue quase sozinho no núcleo depois de 1,6 s. Com um processo a mais ($p = 16$, 32), o artigo é 4× mais rápido que a clássica. *Excesso de processos.* Acima de 32, com `mpi_yield_when_idle`, o nó interno bloqueado cede o núcleo às folhas.

## 8 Mais processos (63 a 1023)

Cada vez que as folhas dobram, o trabalho total cai pela metade, mas crescem o lançamento, os níveis de comunicação e a disputa por CPU (Tab. 5, Fig. 3). O ponto ótimo cresce com $N$. Com $10^4$ o melhor é $p \approx 31$, e 1023 processos ficam mais lentos que o sequencial na clássica. Com $10^5$ o melhor é $p = 127$ ($S = 222$ e 545). Com $10^6$ o melhor é $p = 511$: 0,67 s na clássica ($S = 1483$, $S/32 = 46$) e 0,36 s no artigo ($S = 2724$). Com 1023 os dois pioram: cada um dos 2 nós roda ≈ 512 processos em 16 threads.

## 9 Conclusão

O speed-up superlinear vem do algoritmo: dividir corta o trabalho quadrático. A árvore do artigo, em que todos ordenam, é 4× mais rápida com $p = 2^k$, mas perde a vantagem com $p = 2^k{-}1$. Com $N$ grande, processos além das threads compensam (até 511 com $10^6$).

## Anexo A — Tabelas e gráficos

**Tabela 1.** Tempo (s), speed-up e eficiência por número de processos.

| | | árvore clássica | | | árvore do artigo | | |
|---|---|---|---|---|---|---|---|
| $N$ | $p$ | tempo | $S$ | $E$ | tempo | $S$ | $E$ |
| 10000 | 1 | 0,103 | 0,95 | 0,95 | 0,103 | 0,96 | 0,96 |
| 10000 | 3 | 0,028 | 3,50 | 1,17 | 0,028 | 3,54 | 1,18 |
| 10000 | 7 | 0,017 | 5,71 | 0,82 | 0,008 | 12,34 | 1,76 |
| 10000 | 15 | 0,013 | 7,43 | 0,50 | 0,003 | 28,37 | 1,89 |
| 10000 | 31 | 0,010 | 9,80 | 0,32 | 0,002 | 56,31 | 1,82 |
| 100000 | 1 | 10,061 | 1,00 | 1,00 | 10,080 | 1,00 | 1,00 |
| 100000 | 3 | 2,484 | 4,04 | 1,35 | 2,490 | 4,03 | 1,34 |
| 100000 | 7 | 0,650 | 15,44 | 2,21 | 0,624 | 16,09 | 2,30 |
| 100000 | 15 | 0,186 | 53,96 | 3,60 | 0,166 | 60,32 | 4,02 |
| 100000 | 31 | 0,082 | 122,51 | 3,95 | 0,056 | 180,18 | 5,81 |
| 1000000 | 3 | 249,861 | 3,98 | 1,33 | 250,060 | 3,97 | 1,32 |
| 1000000 | 7 | 64,190 | 15,48 | 2,21 | 62,552 | 15,88 | 2,27 |
| 1000000 | 15 | 16,492 | 60,25 | 4,02 | 16,315 | 60,90 | 4,06 |
| 1000000 | 31 | 6,823 | 145,63 | 4,70 | 4,765 | 208,51 | 6,73 |

**Tabela 2.** Árvore do artigo com $2^k$ processos.

| $N$ | $p$ | tempo | $S$ | $E$ |
|---|---|---|---|---|
| 10000 | 2 | 0,027 | 3,59 | 1,79 |
| 10000 | 4 | 0,008 | 12,65 | 3,16 |
| 10000 | 8 | 0,004 | 25,25 | 3,16 |
| 10000 | 16 | 0,002 | 50,05 | 3,13 |
| 10000 | 32 | 0,002 | 64,93 | 2,03 |
| 100000 | 2 | 2,495 | 4,02 | 2,01 |
| 100000 | 4 | 0,625 | 16,06 | 4,01 |
| 100000 | 8 | 0,174 | 57,75 | 7,22 |
| 100000 | 16 | 0,069 | 145,74 | 9,11 |
| 100000 | 32 | 0,020 | 491,04 | 15,34 |
| 1000000 | 2 | 249,965 | 3,97 | 1,99 |
| 1000000 | 4 | 62,672 | 15,85 | 3,96 |
| 1000000 | 8 | 15,928 | 62,38 | 7,80 |
| 1000000 | 16 | 4,142 | 239,87 | 14,99 |
| 1000000 | 32 | 1,661 | 598,16 | 18,69 |

**Tabela 3.** HT: tempos (s) e ganhos.

| $N$ | versão | $T_{15}$ | $T_{31}$ | 15→31 | HT | físicos | ganho |
|---|---|---|---|---|---|---|---|
| 10000 | arvore | 0,013 | 0,010 | 1,32 | 0,009 | 0,013 | 1,50 |
| 10000 | artigo | 0,003 | 0,002 | 1,99 | 0,002 | 0,002 | 0,91 |
| 100000 | arvore | 0,186 | 0,082 | 2,27 | 0,081 | 0,095 | 1,17 |
| 100000 | artigo | 0,166 | 0,056 | 2,99 | 0,057 | 0,053 | 0,93 |
| 1000000 | arvore | 16,492 | 6,823 | 2,42 | 6,592 | 7,799 | 1,18 |
| 1000000 | artigo | 16,315 | 4,765 | 3,42 | 4,757 | 5,013 | 1,05 |

**Tabela 4.** Carga média por nível, $N = 10^6$ ($p = 15$ e 16): pedaço recebido e ordenado, tempos (s) e ocupação (BS + intercalação sobre o tempo total).

| versão | $p$ | nível | ranks | recebido | ordenado | BS | intercala | espera | ocup. |
|---|---|---|---|---|---|---|---|---|---|
| arvore | 15 | 0 | 1 | 1000000 | 0 | 0,000 | 0,005 | 18,026 | 0,0% |
| arvore | 15 | 1 | 2 | 500000 | 0 | 0,000 | 0,005 | 17,235 | 0,0% |
| arvore | 15 | 2 | 4 | 250000 | 0 | 0,000 | 0,001 | 16,832 | 0,0% |
| arvore | 15 | 3 | 8 | 125000 | 125000 | 16,615 | 0,000 | 0,000 | 92,1% |
| artigo | 15 | 0 | 1 | 1000000 | 62500 | 4,092 | 0,009 | 12,202 | 25,1% |
| artigo | 15 | 1 | 1 | 500000 | 62500 | 4,092 | 0,006 | 12,189 | 25,1% |
| artigo | 15 | 2 | 2 | 250000 | 62500 | 4,092 | 0,002 | 6,098 | 25,1% |
| artigo | 15 | 3 | 4 | 125000 | 78125 | 7,175 | 0,000 | 0,004 | 44,0% |
| artigo | 15 | 4 | 7 | 62500 | 62500 | 4,109 | 0,000 | 0,000 | 25,2% |
| artigo | 16 | 0 | 1 | 1000000 | 62500 | 4,097 | 0,010 | 0,843 | 82,9% |
| artigo | 16 | 1 | 1 | 500000 | 62500 | 4,088 | 0,004 | 0,006 | 82,6% |
| artigo | 16 | 2 | 2 | 250000 | 62500 | 4,508 | 0,002 | 0,002 | 91,0% |
| artigo | 16 | 3 | 4 | 125000 | 62500 | 4,094 | 0,001 | 0,210 | 82,6% |
| artigo | 16 | 4 | 8 | 62500 | 62500 | 4,199 | 0,000 | 0,000 | 84,8% |

**Tabela 5.** Excesso de processos (eficiência sobre as 32 hardware threads).

| | | árvore clássica | | | árvore do artigo | | |
|---|---|---|---|---|---|---|---|
| $N$ | $p$ | tempo | $S$ | $S/32$ | tempo | $S$ | $S/32$ |
| 10000 | 63 | 0,014 | 6,82 | 0,21 | 0,002 | 44,73 | 1,40 |
| 10000 | 127 | 0,016 | 6,12 | 0,19 | 0,005 | 19,83 | 0,62 |
| 10000 | 255 | 0,035 | 2,82 | 0,09 | 0,016 | 5,97 | 0,19 |
| 10000 | 511 | 0,086 | 1,14 | 0,04 | 0,019 | 5,13 | 0,16 |
| 10000 | 1023 | 0,225 | 0,44 | 0,01 | 0,098 | 1,00 | 0,03 |
| 100000 | 63 | 0,062 | 162,11 | 5,07 | 0,027 | 373,46 | 11,67 |
| 100000 | 127 | 0,045 | 222,18 | 6,94 | 0,018 | 545,32 | 17,04 |
| 100000 | 255 | 0,058 | 174,35 | 5,45 | 0,019 | 519,81 | 16,24 |
| 100000 | 511 | 0,118 | 85,30 | 2,67 | 0,038 | 265,28 | 8,29 |
| 100000 | 1023 | 0,337 | 29,78 | 0,93 | 0,121 | 83,00 | 2,59 |
| 1000000 | 63 | 3,235 | 307,18 | 9,60 | 1,688 | 588,76 | 18,40 |
| 1000000 | 127 | 1,740 | 571,05 | 17,85 | 0,735 | 1351,39 | 42,23 |
| 1000000 | 255 | 0,984 | 1009,80 | 31,56 | 0,404 | 2458,60 | 76,83 |
| 1000000 | 511 | 0,670 | 1483,04 | 46,34 | 0,365 | 2723,76 | 85,12 |
| 1000000 | 1023 | 1,395 | 712,26 | 22,26 | 0,701 | 1417,03 | 44,28 |

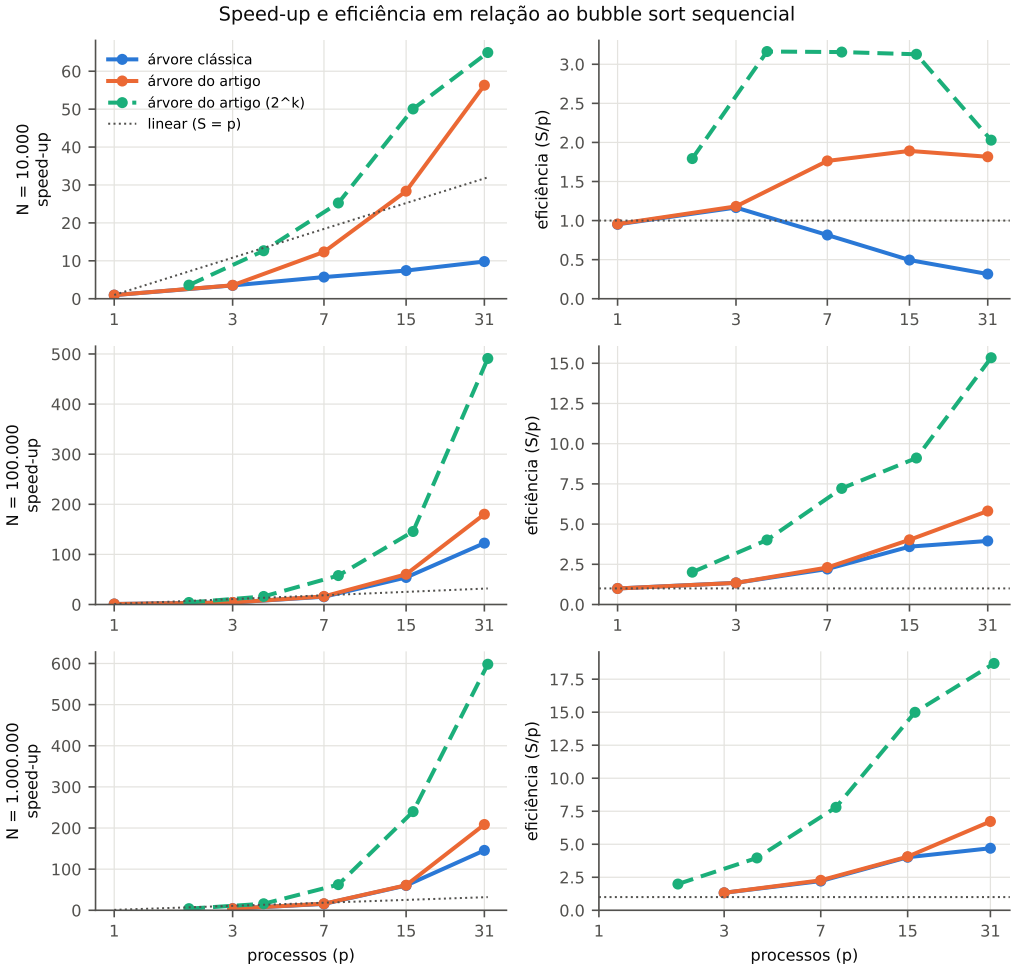



**Figura 1.** Speed-up e eficiência; a linha pontilhada é o ideal linear.

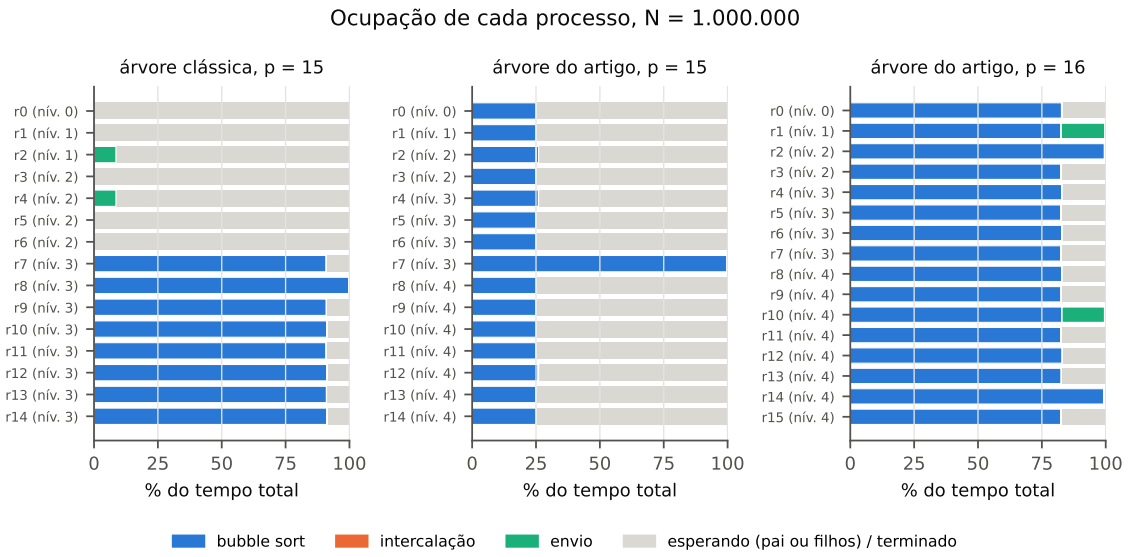



**Figura 2.** Ocupação de cada processo durante a ordenação ($N = 10^6$).

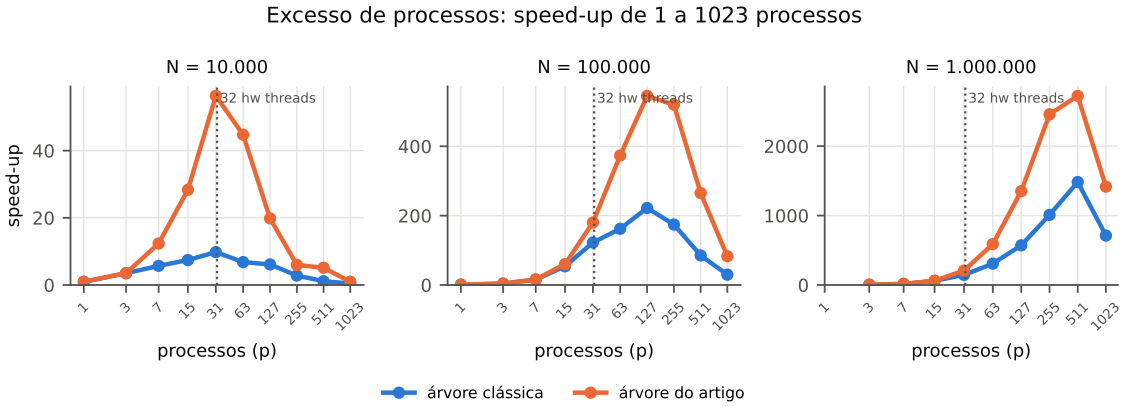



**Figura 3.** Speed-up com excesso de processos.

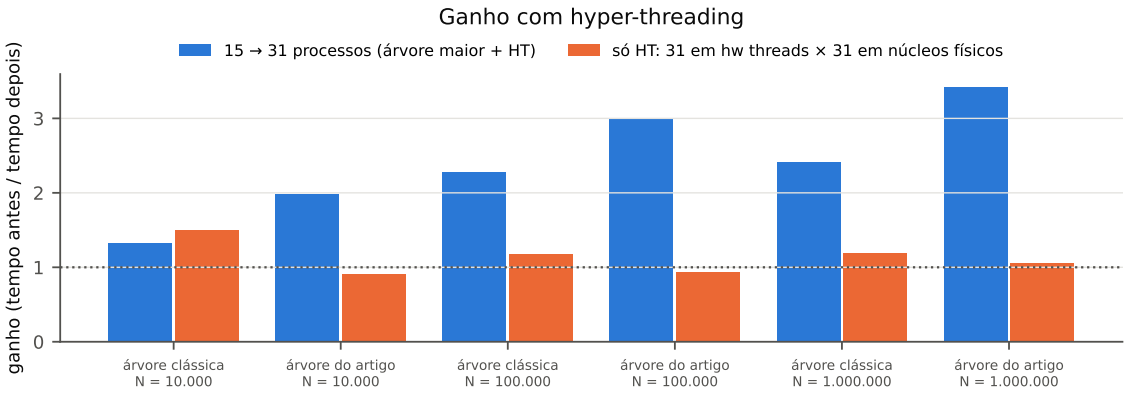



**Figura 4.** Ganho de 15 para 31 processos e ganho isolado do HT.

# Anexo B — Código fonte

## bs.c

```c
/*
 * Trabalho 3 - Programacao Paralela - Grupo 20 (cp32020)
 * Bubble sort por divisao e conquista em MPI.
 *
 * Um unico fonte gera os dois binarios:
 * gcc bs.c -> bs_seq (bubble sort do vetor inteiro, base do speed-up)
 * mpicc -DPARALELO bs.c -> bs_mpi (duas arvores de processos, escolhidas com -v)
 *
 * -v arvore arvore classica (pseudocodigo do enunciado): os nos internos so
 * dividem e intercalam; apenas as folhas ordenam.
 * -v artigo arvore do artigo (Radenski, PDPTA'11): cada processo fica com uma
 * metade e envia a outra a um ajudante (rank + 2^nivel), de modo que
 * todos os processos, inclusive os internos, tambem ordenam.
 * Usa o MESMO bubble sort e a MESMA intercalacao da arvore classica
 * (docs/adr/0001).
 *
 * Opcoes:
 * -n N tamanho do vetor (padrao 40)
 * -v versao arvore | artigo (so no bs_mpi; padrao arvore)
 * -d delta maior pedaco conquistado na arvore classica (padrao: calculado
 * para que a arvore use exatamente np processos; docs/adr/0002)
 * -b arquivo acrescenta ao arquivo uma linha de carga por rank (CSV)
 * -m mostra o vetor antes e depois (so para N pequeno)
 *
 * Saida (stdout, uma linha CSV): versao;processos;n;delta;tempo;ok
 * ok = 1 se o vetor terminou ordenado (como a entrada e N, N-1, ..., 1,
 * o resultado correto e exatamente 1, 2, ..., N).
 */
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
#ifdef PARALELO
#include <mpi.h>
#endif

/* ------------------------------------------------------------------------ */
/* Ordenacao e intercalacao (codigo do enunciado, com correcoes) */
/* ------------------------------------------------------------------------ */

/* Bubble sort do enunciado; para assim que uma passada nao troca nada. */
static void bs(int n, int *vetor)
{
    int c = 0, d, troca, trocou = 1;

    while (c < (n - 1) && trocou) {
        trocou = 0;
        for (d = 0; d < n - c - 1; d++)
            if (vetor[d] > vetor[d + 1]) {
                troca = vetor[d];
                vetor[d] = vetor[d + 1];
                vetor[d + 1] = troca;
                trocou = 1;
            }
        c++;
    }
}

#ifdef PARALELO /* a versao sequencial ordena o vetor inteiro, sem intercalar */
/*
 * Rotina de intercalacao do enunciado: as duas metades ordenadas ficam em
 * vetor[0, tam/2) e vetor[tam/2, tam); devolve um vetor novo com o resultado.
 * Correcao: o limite (i2 == tam, i1 < tam/2) e testado ANTES de ler o vetor;
 * no original, vetor[i2] era lido com i2 == tam (uma posicao alem do fim).
 */
static int *interleaving(const int vetor[], int tam)
{
    int *vetor_auxiliar = malloc(sizeof(int) * (size_t)tam);
    int meio = tam / 2;
    int i1 = 0, i2 = meio, i_aux;

    if (vetor_auxiliar == NULL) {
        fprintf(stderr, "erro: sem memoria para intercalar %d inteiros\n", tam);
        exit(1);
    }
    for (i_aux = 0; i_aux < tam; i_aux++) {
        if (i2 == tam || (i1 < meio && vetor[i1] <= vetor[i2]))
            vetor_auxiliar[i_aux] = vetor[i1++];
        else
            vetor_auxiliar[i_aux] = vetor[i2++];
    }
    return vetor_auxiliar;
}

/* Intercala as duas metades e devolve o resultado no proprio vetor
 * (copia de volta e libera o auxiliar, que no original vazava). */
static void intercala(int vetor[], int tam)
{
    int *vetor_auxiliar = interleaving(vetor, tam);
    memcpy(vetor, vetor_auxiliar, sizeof(int) * (size_t)tam);
    free(vetor_auxiliar);
}
#endif

/* ------------------------------------------------------------------------ */
/* Utilitarios */
/* ------------------------------------------------------------------------ */

static int *aloca(int n)
{
    int *v = malloc(sizeof(int) * (size_t)(n > 0 ? n : 1));
    if (v == NULL) {
        fprintf(stderr, "erro: sem memoria para %d inteiros\n", n);
        exit(1);
    }
```

```c
    return v;
}

/* Pior caso do bubble sort: ordem decrescente N, N-1, ..., 1. */
static void inicializa(int *vetor, int n)
{
    for (int i = 0; i < n; i++)
        vetor[i] = n - i;
}

/* 1 se o vetor e exatamente 1, 2, ..., N (ordenado e sem perda de elementos). */
static int confere(const int *vetor, int n)
{
    for (int i = 0; i < n; i++)
        if (vetor[i] != i + 1)
            return 0;
    return 1;
}

static void mostra(const char *rotulo, const int *vetor, int n)
{
    printf("%s:", rotulo);
    for (int i = 0; i < n; i++)
        printf(" [%03d]", vetor[i]);
    printf("\n");
}

typedef struct {
    int n, delta, mostrar;
    const char *versao, *arquivo_carga;
} Opcoes;

static Opcoes le_opcoes(int argc, char **argv)
{
    Opcoes o = { 40, 0, 0, "arvore", NULL };
    int c;
    while ((c = getopt(argc, argv, "n:v:d:b:m")) != -1) {
        switch (c) {
        case 'n': o.n = atoi(optarg); break;
        case 'v': o.versao = optarg; break;
        case 'd': o.delta = atoi(optarg); break;
        case 'b': o.arquivo_carga = optarg; break;
        case 'm': o.mostrar = 1; break;
        default:
            fprintf(stderr, "uso: %s [-n N] [-v arvore|artigo] [-d delta] "
                            "[-b carga.csv] [-m]\n", argv[0]);
            exit(1);
        }
    }
    if (o.n < 1) {
        fprintf(stderr, "erro: N deve ser positivo\n");
        exit(1);
    }
    return o;
}

#ifndef PARALELO
/* ========================================================================= */
/* Versao sequencial */
/* ========================================================================= */

static double agora(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return (double)t.tv_sec + (double)t.tv_nsec * 1e-9;
}

int main(int argc, char **argv)
{
    Opcoes o = le_opcoes(argc, argv);
    int *vetor = aloca(o.n);

    inicializa(vetor, o.n);
    if (o.mostrar) mostra("Vetor", vetor, o.n);

    double inicio = agora();
    bs(o.n, vetor); /* so a ordenacao e medida */
    double tempo = agora() - inicio;

    if (o.mostrar) mostra("Vetor", vetor, o.n);
    int ok = confere(vetor, o.n);
    printf("seq;1;%d;%d;%.6f;%d\n", o.n, o.n, tempo, ok);
    free(vetor);
    return ok ? 0 : 2;
}

#else
/* ========================================================================= */
/* Versao paralela */
/* ========================================================================= */

#define TAG 1

/*
 * Carga de um rank, para a analise do balanceamento por nivel da arvore.
 * Instantes medidos a partir do MPI_Barrier que antecede a ordenacao.
 */
typedef struct {
    double nivel; /* nivel do rank na arvore (raiz = 0) */
    double filhos; /* quantos processos receberam parte do vetor deste rank */
    double recebido; /* tamanho do pedaco que o rank recebeu */
    double conquistado; /* tamanho do pedaco que o rank ordenou (0: so intercala) */
    double inicio; /* quando o pedaco chegou (raiz: 0) */
    double t_bs; /* tempo no bubble sort (conquista) */
    double t_intercala; /* tempo intercalando */
    double t_envio; /* tempo nos envios (aos filhos e ao pai) */
    double t_espera; /* tempo bloqueado esperando a resposta dos filhos */
    double fim; /* quando o rank terminou seu papel */
} Carga;
#define CAMPOS_CARGA ((int)(sizeof(Carga) / sizeof(double)))

```

```c
static double t0; /* instante comum de partida (logo apos o MPI_Barrier) */

/* ------------------------------ arvore classica ------------------------- */
/*
 * Numeracao de heap: filhos de r sao 2r+1 e 2r+2; pai de r e (r-1)/2.
 * Simula a divisao do vetor para saber quais ranks a arvore usara com este
 * delta. Devolve quantos nos a arvore tem e, em *maior, o maior rank usado.
 */
static int conta_nos(int rank, int tam, int delta, int *maior)
{
    if (rank > *maior) *maior = rank;
    if (tam <= delta) return 1;
    return 1 + conta_nos(2 * rank + 1, tam / 2, delta, maior)
             + conta_nos(2 * rank + 2, tam - tam / 2, delta, maior);
}

static int nivel_heap(int rank)
{
    int nivel = 0;
    while ((1 << (nivel + 1)) - 1 <= rank) nivel++;
    return nivel;
}

/* Executa o papel deste rank na arvore classica. Na raiz, vetor ja contem os
 * dados e, ao final, contem o resultado; nos demais ranks, vetor e NULL. */
static void arvore_classica(int *vetor, int n, int delta, int my_rank, Carga *c)
{
    MPI_Status status;
    int tam_vetor;
    double t;

    c->nivel = nivel_heap(my_rank);

    /* recebo vetor */
    if (my_rank != 0) {
        int pai = (my_rank - 1) / 2;
        MPI_Probe(pai, TAG, MPI_COMM_WORLD, &status); /* nao sou a raiz, tenho pai */
        MPI_Get_count(&status, MPI_INT, &tam_vetor); /* descubro o tamanho */
        vetor = aloca(tam_vetor);
        MPI_Recv(vetor, tam_vetor, MPI_INT, pai, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    } else {
        tam_vetor = n; /* sou a raiz: vetor inteiro */
    }
    c->recebido = tam_vetor;
    c->inicio = MPI_Wtime() - t0;

    /* dividir ou conquistar? */
    if (tam_vetor <= delta) {
        c->conquistado = tam_vetor;
        t = MPI_Wtime();
        bs(tam_vetor, vetor); /* conquisto */
        c->t_bs = MPI_Wtime() - t;
    } else {
        /* divido: metade inicial para a esquerda, metade final para a direita.
         * Correcao: com tam_vetor impar, a direita leva o elemento a mais (no
         * pseudocodigo as duas metades tinham tam_vetor/2 e um elemento sumia). */
        int meio = tam_vetor / 2;
        int esq = 2 * my_rank + 1, dir = 2 * my_rank + 2;
        c->filhos = 2;

        t = MPI_Wtime();
        MPI_Send(&vetor[0], meio, MPI_INT, esq, TAG, MPI_COMM_WORLD);
        MPI_Send(&vetor[meio], tam_vetor - meio, MPI_INT, dir, TAG, MPI_COMM_WORLD);
        c->t_envio += MPI_Wtime() - t;

        /* recebo dos filhos, cada metade de volta ao seu lugar */
        t = MPI_Wtime();
        MPI_Recv(&vetor[0], meio, MPI_INT, esq, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Recv(&vetor[meio], tam_vetor - meio, MPI_INT, dir, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        c->t_espera = MPI_Wtime() - t;

        t = MPI_Wtime();
        intercala(vetor, tam_vetor); /* intercalo o vetor inteiro */
        c->t_intercala = MPI_Wtime() - t;
    }

    /* mando para o pai */
    if (my_rank != 0) {
        t = MPI_Wtime();
        MPI_Send(vetor, tam_vetor, MPI_INT, (my_rank - 1) / 2, TAG, MPI_COMM_WORLD);
        c->t_envio += MPI_Wtime() - t;
        free(vetor);
    }
    c->fim = MPI_Wtime() - t0;
}

/* ------------------------------ arvore do artigo ------------------------ */
/*
 * Nivel em que o rank aparece pela primeira vez na arvore do artigo: o menor
 * nivel l com 2^l > rank (my_topmost_level_mpi do artigo). A raiz esta no 0.
 */
static int nivel_topo(int rank)
{
    int nivel = 0;
    while ((1 << nivel) <= rank) nivel++;
    return nivel;
}

/* mergesort_parallel_mpi do artigo, com bubble sort no caso base. Enquanto
 * houver ajudante (rank + 2^nivel < np), manda a metade final a ele e segue
 * dividindo a metade inicial; sem ajudante, conquista o que sobrou. */
static void artigo_recursivo(int *vetor, int tam, int nivel, int my_rank, int np, Carga *c)
{
    int ajudante = my_rank + (1 << nivel);
    double t;

    if (ajudante >= np) { /* sem mais processos: conquisto */
        c->conquistado = tam;
        t = MPI_Wtime();
        bs(tam, vetor);
        c->t_bs = MPI_Wtime() - t;
        return;
```

```
    }

    int meio = tam / 2;
    c->filhos += 1;

    /* Metade final ao ajudante. Envio BLOQUEANTE, diferente do MPI_Isend do
     * artigo: para mensagens grandes o Open MPI so transfere os dados quando
     * quem enviou volta a chamar o MPI, e o bubble sort nao chama nenhuma
     * funcao MPI. Com Isend o ajudante so recebia a metade dele depois que
     * este rank terminava de ordenar a sua (no ensaio com N=20000 e p=2:
     * 0,202 s com Isend contra ~0,10 s esperados). O ajudante ja esta parado
     * no MPI_Probe, entao o Send termina assim que os dados chegam. */
    t = MPI_Wtime();
    MPI_Send(&vetor[meio], tam - meio, MPI_INT, ajudante, TAG, MPI_COMM_WORLD);
    c->t_envio += MPI_Wtime() - t;

    artigo_recursivo(vetor, meio, nivel + 1, my_rank, np, c);

    /* metade final ordenada de volta */
    t = MPI_Wtime();
    MPI_Recv(&vetor[meio], tam - meio, MPI_INT, ajudante, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    c->t_espera += MPI_Wtime() - t;

    t = MPI_Wtime();
    intercala(vetor, tam);
    c->t_intercala += MPI_Wtime() - t;
}

static void arvore_artigo(int *vetor, int n, int my_rank, int np, Carga *c)
{
    int nivel = nivel_topo(my_rank);
    int tam = n, pai = 0;
    double t;

    c->nivel = nivel;
    if (my_rank != 0) {
        /* como no run_helper_mpi do artigo: o pai e quem mandar primeiro */
        MPI_Status status;
        MPI_Probe(MPI_ANY_SOURCE, TAG, MPI_COMM_WORLD, &status);
        MPI_Get_count(&status, MPI_INT, &tam);
        pai = status.MPI_SOURCE;
        vetor = aloca(tam);
        MPI_Recv(vetor, tam, MPI_INT, pai, TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
    c->recebido = tam;
    c->inicio = MPI_Wtime() - t0;

    artigo_recursivo(vetor, tam, nivel, my_rank, np, c);

    if (my_rank != 0) {
        t = MPI_Wtime();
        MPI_Send(vetor, tam, MPI_INT, pai, TAG, MPI_COMM_WORLD);
        c->t_envio += MPI_Wtime() - t;
        free(vetor);
    }
    c->fim = MPI_Wtime() - t0;
}

/* ------------------------------ programa principal ---------------------- */

static void grava_carga(const char *arquivo, const char *versao, int np, int n,
                        const Carga *todas)
{
    FILE *f = fopen(arquivo, "a");
    if (f == NULL) {
        perror(arquivo);
        return;
    }
    for (int r = 0; r < np; r++) {
        const Carga *c = &todas[r];
        fprintf(f, "%s;%d;%d;%d;%d;%d;%d;%d;%.6f;%.6f;%.6f;%.6f;%.6f;%.6f\n",
                versao, np, n, r, (int)c->nivel, (int)c->filhos, (int)c->recebido,
                (int)c->conquistado,
                c->inicio, c->t_bs, c->t_intercala, c->t_envio, c->t_espera, c->fim);
    }
    fclose(f);
}

int main(int argc, char **argv)
{
    int my_rank, np;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &np);

    Opcoes o = le_opcoes(argc, argv);
    int classica = strcmp(o.versao, "arvore") == 0;
    if (!classica && strcmp(o.versao, "artigo") != 0) {
        if (my_rank == 0) fprintf(stderr, "erro: versao deve ser arvore ou artigo\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    /* Delta da arvore classica: com folhas = (np+1)/2, ceil(N/folhas) faz a
     * arvore descer exatamente ate usar np processos. Todos os ranks fazem a
     * mesma conta; se a arvore nao usar exatamente os ranks 0..np-1, aborta
     * em vez de deixar processos esperando para sempre por um vetor. */
    int delta = o.n;
    if (classica) {
        int folhas = (np + 1) / 2;
        delta = o.delta > 0 ? o.delta : (o.n + folhas - 1) / folhas;
        int maior = 0;
        int nos = conta_nos(0, o.n, delta, &maior);
        if (nos != np || maior != np - 1) {
            if (my_rank == 0)
                fprintf(stderr, "erro: com N=%d e delta=%d a arvore classica usa %d "
                        "processos (maior rank %d), mas ha %d. Use np = 2^k - 1 "
                        "ou ajuste -d.\n", o.n, delta, nos, maior, np);
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
    }

```

```
    int *vetor = NULL;
    if (my_rank == 0) { /* sou a raiz: gero o vetor */
        vetor = aloca(o.n);
        inicializa(vetor, o.n);
        if (o.mostrar) mostra("Vetor", vetor, o.n);
    }

    Carga carga;
    memset(&carga, 0, sizeof carga);

    MPI_Barrier(MPI_COMM_WORLD); /* todos partem juntos */
    t0 = MPI_Wtime();
    if (classica)
        arvore_classica(vetor, o.n, delta, my_rank, &carga);
    else
        arvore_artigo(vetor, o.n, my_rank, np, &carga);
    double tempo = MPI_Wtime() - t0; /* so vale na raiz */

    /* Fora da medicao: a raiz junta a carga de todos os ranks. Todos os ranks
     * leem as mesmas opcoes, entao todos entram (ou nao) no MPI_Gather. */
    Carga *todas = NULL;
    if (o.arquivo_carga) {
        if (my_rank == 0)
            todas = malloc(sizeof(Carga) * (size_t)np);
        MPI_Gather(&carga, CAMPOS_CARGA, MPI_DOUBLE,
                   todas, CAMPOS_CARGA, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    }

    int ok = 1;
    if (my_rank == 0) { /* sou a raiz: mostro e confiro */
        if (o.mostrar) mostra("Vetor", vetor, o.n);
        ok = confere(vetor, o.n);
        printf("%s;%d;%d;%d;%.6f;%d\n", o.versao, np, o.n, delta, tempo, ok);
        fflush(stdout);
        if (todas) {
            grava_carga(o.arquivo_carga, o.versao, np, o.n, todas);
            free(todas);
        }
        free(vetor);
    }

    MPI_Finalize();
    return ok ? 0 : 2;
}
#endif
```

# medir.sh e job.sh

```
#!/usr/bin/env bash
# Coleta os tempos do T3 para UM tamanho de vetor: ./medir.sh N
# Deve rodar dentro da alocacao de 2 nos exclusivos do job.sh. Gera em $RES:
#
# tempos_N.csv sequencial + arvore classica + arvore do artigo, por np
# (1..31, 2..32 no artigo, e 63..1023 nas duas arvores)
# carga_N.csv uma linha por rank de cada execucao -> balanceamento por nivel
# ht_N.csv 31 processos com HT x os mesmos 31 so nos nucleos fisicos
#
# Cada tempo e a MEDIANA de ate REPS execucoes. Se uma execucao passa de
# LIMITE_REPS segundos, as demais repeticoes daquele ponto sao puladas (o
# sequencial de 1M leva ~74 min): a coluna reps diz quantas valeram.
set -euo pipefail
cd "$(dirname "$0")"

N=${1:?uso: ./medir.sh N}
REPS=${REPS:-3}
LIMITE_REPS=${LIMITE_REPS:-600}
TEMPO_MAX=${TEMPO_MAX:-7200} # aborta uma execucao travada (seq 1M ~ 74 min)
PROCS_ARVORE=${PROCS_ARVORE:-"3 7 15 31"}
PROCS_ARTIGO=${PROCS_ARTIGO:-"2 3 4 7 8 15 16 31 32"}
PROCS_MUITOS=${PROCS_MUITOS:-"63 127 255 511 1023"}
NP_HT=${NP_HT:-31}
RES=${RES:-resultados}
LOCAL=${LOCAL:-0} # 1 = teste fora do cluster, sem Slurm

# np=1 na versao MPI e identico ao sequencial + overhead do MPI: so vale a
# pena para os tamanhos em que custa pouco (com 1M seriam outros 74 min).
if [ "$N" -lt 1000000 ]; then
    PROCS_ARVORE="1 $PROCS_ARVORE"; PROCS_ARTIGO="1 $PROCS_ARTIGO"
fi

# ------------------------------------------------------------ lancador -----
if [ "$LOCAL" = 1 ]; then
    NUC=64; HW=64; PRIMARIAS=""
else
    NUC=$(LC_ALL=C lscpu | awk -F: '/^Core\(s\) per socket/{c=$2} /^Socket\(s\)/{s=$2} END{print c*s}')
    HW=$(nproc --all)
    # Uma hardware thread por nucleo fisico (a primeira de cada par HT).
    PRIMARIAS=$(lscpu -p=CPU,CORE,SOCKET | grep -v '^#' | awk -F, '!v[$2","$3]++{print $1}' | paste -sd,)
fi

# lanca NP programa args... (2 nos, round-robin: metade dos ranks em cada)
# ate NUC ranks por no : um rank por nucleo fisico
# ate HW ranks por no : um rank por hardware thread (usa o HT)
# acima disso : excesso de processos; sem fixacao e cedendo a CPU
# quando bloqueado, senao a espera ativa dos nos
# internos rouba o processador das folhas.
lanca() {
    local np=$1; shift
    if [ -n "${MPIRUN:-}" ]; then timeout "$TEMPO_MAX" $MPIRUN -n "$np" "$@"; return; fi
    if [ "$LOCAL" = 1 ]; then timeout "$TEMPO_MAX" mpirun --oversubscribe -n "$np" "$@"; return; fi
    local por_no=$(( (np + 1) / 2 )) extra=""
    if [ "$por_no" -gt "$HW" ]; then extra="--bind-to none --mca mpi_yield_when_idle 1"
    elif [ "$por_no" -gt "$NUC" ]; then extra="--use-hwthread-cpus"
    fi
    timeout "$TEMPO_MAX" mpirun --oversubscribe --map-by node $extra -n "$np" "$@"
}

# Mesmos ranks, mas confinados a uma hardware thread por nucleo fisico: dois
# ranks por nucleo se revezam (fatia de tempo) em vez de rodar em paralelo no
# HT. Comparado com lanca, isola o ganho do HT (mesma arvore, mesmo trabalho).
```

```
lanca_fisico() {
    local np=$1; shift
    timeout "$TEMPO_MAX" mpirun --oversubscribe --map-by node --bind-to none \
        --mca mpi_yield_when_idle 1 -n "$np" taskset -c "$PRIMARIAS" "$@"
}

# mediana comando... -> linha da mediana + ;reps;tmin;tmax
# Falha (sem saida, status 1) se alguma execucao falhar ou passar de
# TEMPO_MAX segundos: um ponto travado nao derruba o resto da bateria.
mediana() {
    local i linha t saida=()
    for ((i = 0; i < REPS; i++)); do
        linha=$("$@" | tail -1) || return 1
        [ -n "$linha" ] || return 1
        saida+=("$linha")
        t=$(echo "$linha" | cut -d';' -f5)
        awk -v t="$t" -v l="$LIMITE_REPS" 'BEGIN{exit !(t > l)}' && break
    done
    printf '%s\n' "${saida[@]}" | sort -t';' -k5 -g |
        awk -F';' '{l[NR]=$0; t[NR]=$5}
             END {print l[int((NR+1)/2)] ";" NR ";" t[1] ";" t[NR]}'
}

CAB='versao;processos;n;delta;tempo;ok;reps;tmin;tmax'
CAB_CARGA='versao;processos;n;rank;nivel;filhos;recebido;conquistado;inicio;bs;intercala;envio;espera;fim'
mkdir -p "$RES"
TEMPOS=$RES/tempos_$N.csv; CARGA=$RES/carga_$N.csv; HT=$RES/ht_$N.csv
echo "$CAB" > "$TEMPOS"; echo "$CAB_CARGA" > "$CARGA"; echo "modo;$CAB" > "$HT"
# mede ARQUIVO PREFIXO ROTULO comando...: acrescenta a mediana ao arquivo
# (com PREFIXO; antes dele, se nao vazio) e mostra tempo, ok e repeticoes.
mede() {
    local arq=$1 pre=$2 rot=$3 linha; shift 3
    if linha=$(mediana "$@"); then
        echo "$pre$linha" >> "$arq"
        echo " $rot: $(echo "$linha" | cut -d';' -f5-7 --output-delimiter=' ')"
    else
        echo " $rot: FALHOU (erro ou mais de ${TEMPO_MAX}s)"
    fi
}

echo "N=$N | nucleos fisicos/no: $NUC | hw threads/no: $HW | reps: $REPS (limite ${LIMITE_REPS}s)"
echo " (colunas: tempo ok repeticoes)"

# ------------------------------------------------------------ sequencial ----
echo ">> sequencial"
mede "$TEMPOS" "" seq timeout "$TEMPO_MAX" ./bs_seq -n "$N"

# ------------------------------------------------------------------ arvores ----
# -b grava a carga de cada rank em toda execucao: o MPI_Gather acontece depois
# da medicao, entao nao altera o tempo. A analise tira a media das repeticoes.
roda() { # versao np
    mede "$TEMPOS" "" "$1 p=$2" lanca "$2" ./bs_mpi -v "$1" -n "$N" -b "$CARGA"
}
echo ">> arvore classica"; for np in $PROCS_ARVORE; do roda arvore "$np"; done
echo ">> arvore do artigo"; for np in $PROCS_ARTIGO; do roda artigo "$np"; done
echo ">> excesso de processos"
for np in $PROCS_MUITOS; do roda arvore "$np"; roda artigo "$np"; done

# ------------------------------------------------------------------- HT ----
if [ "$LOCAL" != 1 ]; then
    echo ">> HT: $NP_HT processos com HT x so nucleos fisicos"
    for v in arvore artigo; do
        mede "$HT" "ht;" "$v com HT" lanca "$NP_HT" ./bs_mpi -v $v -n "$N"
        mede "$HT" "fisico;" "$v so fisicos" lanca_fisico "$NP_HT" ./bs_mpi -v $v -n "$N"
    done
fi

echo "pronto: $TEMPOS $CARGA $HT"
```

```
#!/usr/bin/env bash
#SBATCH --job-name=bs-mpi-g20
#SBATCH --nodes=2
#SBATCH --exclusive
#SBATCH --time=01:00:00
#SBATCH --output=slurm-%j.out
#
# Um job por tamanho de vetor, em 2 nos exclusivos da atlantica (8 nucleos /
# 16 threads HT cada). O tempo muda muito com N (bubble sort e quadratico),
# entao passe --time na linha de comando:
#
# sbatch --time=00:30:00 job.sh 10000
# sbatch --time=01:00:00 job.sh 100000
# sbatch --time=05:00:00 job.sh 1000000 (sequencial sozinho ~74 min)
#
# Se o sbatch reclamar de particao ou de --time acima do limite, veja a saida
# do job-teste.sh (sinfo / scontrol show partition) e ajuste.
# acompanhar: squeue -u $USER ; tail -f slurm-*.out
set -euo pipefail
cd "$SLURM_SUBMIT_DIR"
# 1023 processos em 2 nos = ~512 por no; o mpirun abre alguns descritores por
# processo (saida de cada um) e o limite padrao de 1024 nao basta.
ulimit -n "$(ulimit -Hn)" 2>/dev/null || true
N=${1:?uso: sbatch job.sh N}

mkdir -p resultados
{ echo "nos: $SLURM_JOB_NODELIST"; LC_ALL=C lscpu; free -g; gcc --version | head -1;
  mpirun --version 2>&1 | head -1; } > resultados/maquina.txt 2>&1

make
./medir.sh "$N"

# Analisa o que ja houver (os outros tamanhos podem nao ter rodado ainda).
python3 analisar.py | tee resultados/tabelas.txt || true
if python3 -c "import matplotlib" 2>/dev/null; then
    python3 graficos.py || true
else
    echo "matplotlib ausente: rode 'python3 graficos.py' na maquina local"
fi
```

# analisar.py

```python
#!/usr/bin/env python3
"""Tabelas do T3 a partir de resultados/ (ou do diretorio dado).

    python3 analisar.py [dir]

Le tempos_N.csv, carga_N.csv e ht_N.csv de cada tamanho N ja medido e
imprime: escala (speed-up e eficiencia), excesso de processos, HT e
balanceamento por nivel da arvore. Grava tambem tab_*.tex, que o relatorio
inclui sozinho. So usa a biblioteca padrao (o no do cluster pode nao ter pandas).

Definicoes (decididas no planejamento):
  S(p) = T_seq / T(p)
  E(p) = S(p) / p para p <= 32 (conta todos os processos, inclusive
                                os nos internos ociosos: e isso que se quer ver)
  E_hw = S(p) / 32 para p > 32: ha mais processos que as 32 hardware
                                threads dos 2 nos, entao S/p perde o sentido.
"""
import csv
import glob
import os
import re
import sys
from collections import defaultdict
from statistics import mean

HW_THREADS = 32 # 2 nos x 16 hardware threads
PROCS_BASE = [1, 3, 7, 15, 31]
PROCS_2K = [2, 4, 8, 16, 32]
PROCS_CARGA = [7, 15, 31]


def le_csv(caminho):
    with open(caminho, newline="") as f:
        return list(csv.DictReader(f, delimiter=";"))


def tamanhos(d):
    """Valores de N com tempos medidos, em ordem crescente."""
    ns = []
    for c in glob.glob(os.path.join(d, "tempos_*.csv")):
        m = re.search(r"tempos_(\d+)\.csv$", c)
        if m:
            ns.append(int(m.group(1)))
    return sorted(ns)


def carrega(d):
    """{N: {"seq": T, ("arvore"|"artigo", p): linha}} com tempo em float."""
    dados = {}
    for n in tamanhos(d):
        pts = {}
        for l in le_csv(os.path.join(d, f"tempos_{n}.csv")):
            l["tempo"] = float(l["tempo"])
            l["processos"] = int(l["processos"])
            if l["ok"] != "1":
                print(f"AVISO: vetor NAO ordenado em {l['versao']} p={l['processos']} N={n}")
            if l["versao"] == "seq":
                pts["seq"] = l["tempo"]
            else:
                pts[(l["versao"], l["processos"])] = l
        dados[n] = pts
    return dados


def speedup(pts, versao, p):
    """(tempo, S, E) ou None se o ponto nao existe."""
    l = pts.get((versao, p))
    if l is None or "seq" not in pts:
        return None
    s = pts["seq"] / l["tempo"]
    return l["tempo"], s, s / (p if p <= HW_THREADS else HW_THREADS)


def carga_por_rank(d, n, versao, p):
    """Media, sobre as repeticoes, da carga de cada rank: lista indexada por rank."""
    caminho = os.path.join(d, f"carga_{n}.csv")
    if not os.path.exists(caminho):
        return []
    soma = defaultdict(list)
    for l in le_csv(caminho):
        if l["versao"] == versao and int(l["processos"]) == p:
            soma[int(l["rank"])].append(l)
    ranks = []
    for r in sorted(soma):
        ls = soma[r]
        reg = {k: mean(float(x[k]) for x in ls)
               for k in ("recebido", "conquistado", "inicio", "bs", "intercala", "envio", "espera", "fim")}
        reg.update(rank=r, nivel=int(ls[0]["nivel"]), filhos=int(ls[0]["filhos"]))
        ranks.append(reg)
    return ranks


def por_nivel(ranks):
    """Agrega a carga por nivel: ranks, pedaco medio, bs medio/max, espera, ocupacao.
    Ocupacao = (bs + intercala) / tempo total da arvore (o fim da raiz)."""
    if not ranks:
        return []
    total = ranks[0]["fim"]
    niveis = defaultdict(list)
    for r in ranks:
        niveis[r["nivel"]].append(r)
    linhas = []
    for nv in sorted(niveis):
        rs = niveis[nv]
        linhas.append(dict(
            nivel=nv, ranks=len(rs),
            recebido=mean(r["recebido"] for r in rs),
            conquistado=mean(r["conquistado"] for r in rs),
            bs_med=mean(r["bs"] for r in rs), bs_max=max(r["bs"] for r in rs),
```

```
            intercala=mean(r["intercala"] for r in rs),
            espera=mean(r["espera"] for r in rs),
            ocupacao=mean((r["bs"] + r["intercala"]) / total for r in rs) if total else 0))
    return linhas


def desbalanceamento(ranks):
    """max/media do tempo de bubble sort entre os ranks que conquistaram (1 = perfeito)."""
    bs = [r["bs"] for r in ranks if r["bs"] > 0]
    return max(bs) / mean(bs) if bs else float("nan")


def carrega_ht(d, n):
    caminho = os.path.join(d, f"ht_{n}.csv")
    if not os.path.exists(caminho):
        return {}
    return {(l["modo"], l["versao"]): float(l["tempo"]) for l in le_csv(caminho)}


# ---------------------------------------------------------------------- saida ----

def fmt(x, casas=2):
    return "--" if x is None else f"{x:.{casas}f}".replace(".", ",")


def grava_tex(d, nome, linhas):
    with open(os.path.join(d, f"{nome}.tex"), "w") as f:
        f.write("\n".join(linhas) + "\n")


def main():
    d = sys.argv[1] if len(sys.argv) > 1 else "resultados"
    dados = carrega(d)
    if not dados:
        sys.exit(f"nenhum tempos_N.csv em {d}/")

    tex_escala, tex_2k, tex_muitos, tex_ht, tex_carga = [], [], [], [], []

    for n, pts in dados.items():
        print(f"\n================ N = {n:,} ================".replace(",", "."))
        print(f"sequencial: {fmt(pts.get('seq'), 3)} s")

        # Escala 1..31: as duas arvores lado a lado
        print("\n p | arvore: tempo S E | artigo: tempo S E")
        for p in PROCS_BASE:
            a, b = speedup(pts, "arvore", p), speedup(pts, "artigo", p)
            if a is None and b is None:
                continue
            ca = (fmt(a[0], 3), fmt(a[1]), fmt(a[2])) if a else ("--",) * 3
            cb = (fmt(b[0], 3), fmt(b[1]), fmt(b[2])) if b else ("--",) * 3
            print(f"{p:3d} | {ca[0]:>12} {ca[1]:>6} {ca[2]:>6} | {cb[0]:>12} {cb[1]:>6} {cb[2]:>6}")
            tex_escala.append(f"{n} & {p} & {' & '.join(ca)} & {' & '.join(cb)} \\\\")

        # Arvore do artigo com 2^k processos (todos os ranks com pedacos iguais)
        print("\n p | artigo 2^k: tempo S E")
        for p in PROCS_2K:
            b = speedup(pts, "artigo", p)
            if b:
                print(f"{p:3d} | {fmt(b[0], 3):>16} {fmt(b[1]):>6} {fmt(b[2]):>6}")
                tex_2k.append(f"{n} & {p} & {fmt(b[0], 3)} & {fmt(b[1])} & {fmt(b[2])} \\\\")

        # Excesso de processos
        muitos = sorted({k[1] for k in pts if isinstance(k, tuple) and k[1] > HW_THREADS})
        if muitos:
            print("\n p | arvore: tempo S S/32 | artigo: tempo S S/32")
            for p in muitos:
                a, b = speedup(pts, "arvore", p), speedup(pts, "artigo", p)
                ca = (fmt(a[0], 3), fmt(a[1]), fmt(a[2])) if a else ("--",) * 3
                cb = (fmt(b[0], 3), fmt(b[1]), fmt(b[2])) if b else ("--",) * 3
                print(f"{p:6d} | {ca[0]:>12} {ca[1]:>6} {ca[2]:>5} | {cb[0]:>12} {cb[1]:>6} {cb[2]:>5}")
                tex_muitos.append(f"{n} & {p} & {' & '.join(ca)} & {' & '.join(cb)} \\\\")

        # HT: (a) 15 x 31 processos; (b) os mesmos 31 com HT x so nucleos fisicos
        ht = carrega_ht(d, n)
        print("\nHT | T(15) T(31) 15->31 | 31 com HT 31 so fis. ganho HT")
        for v in ("arvore", "artigo"):
            t15, t31 = pts.get((v, 15)), pts.get((v, 31))
            g1 = t15["tempo"] / t31["tempo"] if t15 and t31 else None
            th, tf = ht.get(("ht", v)), ht.get(("fisico", v))
            g2 = tf / th if th and tf else None
            linha = (fmt(t15 and t15["tempo"], 3), fmt(t31 and t31["tempo"], 3), fmt(g1),
                     fmt(th, 3), fmt(tf, 3), fmt(g2))
            print(f"{v:8} | " + " ".join(f"{c:>8}" for c in linha))
            tex_ht.append(f"{n} & {v} & {' & '.join(linha)} \\\\")

        # Balanceamento por nivel
        for v, procs in (("arvore", PROCS_CARGA), ("artigo", PROCS_CARGA + [8, 16, 32])):
            for p in procs:
                ranks = carga_por_rank(d, n, v, p)
                if not ranks:
                    continue
                print(f"\ncarga {v} p={p}: desbalanceamento do bubble sort (max/media) = "
                      f"{fmt(desbalanceamento(ranks))}")
                print(" nivel ranks recebido conquist. bs med bs max intercala espera ocupacao")
                for l in por_nivel(ranks):
                    print(f" {l['nivel']:5d} {l['ranks']:5d} {l['recebido']:8.0f} {l['conquistado']:9.0f} "
                          f"{l['bs_med']:9.3f} {l['bs_max']:9.3f} {l['intercala']:10.4f} "
                          f"{l['espera']:9.3f} {100 * l['ocupacao']:8.1f}%")
                    if p in (15, 16):
                        tex_carga.append(
                            f"{n} & {v} & {p} & {l['nivel']} & {l['ranks']} & "
                            f"{l['recebido']:.0f} & {l['conquistado']:.0f} & {fmt(l['bs_med'], 3)} & "
                            f"{fmt(l['intercala'], 3)} & {fmt(l['espera'], 3)} & "
                            f"{fmt(100 * l['ocupacao'], 1)}\\% \\\\")

    grava_tex(d, "tab_escala", tex_escala)
    grava_tex(d, "tab_2k", tex_2k)
    grava_tex(d, "tab_muitos", tex_muitos)
    grava_tex(d, "tab_ht", tex_ht)
    grava_tex(d, "tab_carga", tex_carga)


```

```
if __name__ == "__main__":
    main()
```

# graficos.py

```
#!/usr/bin/env python3
"""Figuras do relatorio do T3 (PDF vetorial + PNG para conferir na tela).

    python3 graficos.py [dir]

  grafico_escala.pdf speed-up e eficiencia x p (1..32), um par de paineis por N
  grafico_carga.pdf ocupacao de cada rank ao longo da execucao (p = 15/16)
  grafico_muitos.pdf speed-up x p de 1 a 1023 (excesso de processos)
  grafico_ht.pdf ganho de 15 -> 31 processos e ganho isolado do HT

Usa as mesmas contas do analisar.py. Cores: paleta categorica de referencia
(slots fixos: arvore = azul, artigo = laranja, artigo 2^k = verde-agua), sempre
com legenda; speed-up e eficiencia em paineis separados (nunca eixo duplo).
"""
import os
import sys

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

import analisar as A

COR = {"arvore": "#2a78d6", "artigo": "#eb6834", "artigo2k": "#1baf7a",
       "intercala": "#eb6834", "envio": "#1baf7a"}
NOME = {"arvore": "árvore clássica", "artigo": "árvore do artigo",
        "artigo2k": "árvore do artigo (2^k)"}
TINTA, TINTA2, GRADE, OCIOSO = "#0b0b0b", "#52514e", "#e4e3df", "#d9d8d3"

plt.rcParams.update({
    "font.size": 8, "axes.edgecolor": TINTA2, "axes.labelcolor": TINTA,
    "xtick.color": TINTA2, "ytick.color": TINTA2, "axes.grid": True,
    "grid.color": GRADE, "grid.linewidth": 0.6, "axes.spines.top": False,
    "axes.spines.right": False, "legend.frameon": False, "lines.linewidth": 2,
    "lines.markersize": 4.5, "savefig.bbox": "tight",
})


def salva(fig, d, nome):
    fig.savefig(os.path.join(d, nome + ".pdf"))
    fig.savefig(os.path.join(d, nome + ".png"), dpi=150)
    plt.close(fig)
    print("gerado", os.path.join(d, nome + ".pdf"))


def serie(pts, versao, procs):
    xs, s, e = [], [], []
    for p in procs:
        r = A.speedup(pts, versao, p)
        if r:
            xs.append(p); s.append(r[1]); e.append(r[2])
    return xs, s, e


def rotulo_n(n):
    return f"N = {n:,}".replace(",", ".")


def fig_escala(d, dados):
    ns = list(dados)
    fig, eixos = plt.subplots(len(ns), 2, figsize=(7.2, 2.3 * len(ns)), squeeze=False)
    for i, n in enumerate(ns):
        pts = dados[n]
        curvas = [("arvore", "arvore", A.PROCS_BASE), ("artigo", "artigo", A.PROCS_BASE),
                  ("artigo2k", "artigo", A.PROCS_2K)]
        for chave, versao, procs in curvas:
            xs, s, e = serie(pts, versao, procs)
            if not xs:
                continue
            estilo = dict(color=COR[chave], marker="o", label=NOME[chave])
            if chave == "artigo2k":
                estilo.update(linestyle=(0, (4, 2)))
            eixos[i][0].plot(xs, s, **estilo)
            eixos[i][1].plot(xs, e, **estilo)
        eixos[i][0].plot([1, 32], [1, 32], color=TINTA2, lw=1, ls=":", label="linear (S = p)")
        eixos[i][1].axhline(1, color=TINTA2, lw=1, ls=":")
        eixos[i][0].set_ylabel(f"{rotulo_n(n)}\nspeed-up")
        eixos[i][1].set_ylabel("eficiência (S/p)")
        for ax in eixos[i]:
            ax.set_xscale("log", base=2)
            # so a serie 2^k-1 rotulada: 15|16 e 31|32 se sobrepoem em escala log
            ax.set_xticks([1, 3, 7, 15, 31])
            ax.set_xticklabels(["1", "3", "7", "15", "31"])
            ax.minorticks_off()
            ax.set_ylim(bottom=0)
    for ax in eixos[-1]:
        ax.set_xlabel("processos (p)")
    eixos[0][0].legend(loc="upper left", fontsize=7)
    fig.suptitle("Speed-up e eficiência em relação ao bubble sort sequencial", color=TINTA)
    fig.tight_layout()
    salva(fig, d, "grafico_escala")


def fig_carga(d, dados):
    n = max(dados) # o maior N medido mostra melhor o desbalanceamento
    casos = [("arvore", 15), ("artigo", 15), ("artigo", 16)]
    casos = [(v, p, A.carga_por_rank(d, n, v, p)) for v, p in casos]
    casos = [c for c in casos if c[2]]
    if not casos:
        return
    fig, eixos = plt.subplots(1, len(casos), figsize=(7.2, 3.4), squeeze=False, sharex=True)
    for ax, (v, p, ranks) in zip(eixos[0], casos):
        total = ranks[0]["fim"]
        ordem = sorted(ranks, key=lambda r: (r["nivel"], r["rank"]))
        for y, r in enumerate(ordem):
```

```
            partes = [(r["bs"], COR["arvore"]), (r["intercala"], COR["intercala"]),
                      (r["envio"], COR["envio"])]
            esq = 0.0
            ax.barh(y, 100, color=OCIOSO, height=0.75, linewidth=0) # fundo = esperando
            for t, cor in partes:
                w = 100 * t / total
                ax.barh(y, w, left=esq, color=cor, height=0.75, edgecolor="white", linewidth=0.5)
                esq += w
        ax.set_yticks(range(len(ordem)))
        ax.set_yticklabels([f"r{r['rank']} (nív. {r['nivel']})" for r in ordem], fontsize=6)
        ax.invert_yaxis()
        ax.set_xlim(0, 100)
        ax.grid(axis="y", visible=False)
        ax.set_title(f"{NOME[v]}, p = {p}", fontsize=8, color=TINTA)
        ax.set_xlabel("% do tempo total")
    from matplotlib.patches import Patch
    leg = [Patch(color=COR["arvore"], label="bubble sort"),
           Patch(color=COR["intercala"], label="intercalação"),
           Patch(color=COR["envio"], label="envio"),
           Patch(color=OCIOSO, label="esperando (pai ou filhos) / terminado")]
    fig.legend(handles=leg, loc="lower center", ncol=4, fontsize=7, bbox_to_anchor=(0.5, -0.04))
    fig.suptitle(f"Ocupação de cada processo, {rotulo_n(n)}", color=TINTA)
    fig.tight_layout(rect=(0, 0.04, 1, 1))
    salva(fig, d, "grafico_carga")


def fig_muitos(d, dados):
    ns = list(dados)
    fig, eixos = plt.subplots(1, len(ns), figsize=(7.2, 2.5), squeeze=False)
    for ax, n in zip(eixos[0], ns):
        pts = dados[n]
        procs = sorted({k[1] for k in pts if isinstance(k, tuple)})
        for chave in ("arvore", "artigo"):
            ps = [p for p in procs if p == 1 or (p + 1) & p == 0] # 1, 3, 7, ..., 1023
            xs, s, _ = serie(pts, chave, ps)
            if xs:
                ax.plot(xs, s, color=COR[chave], marker="o", label=NOME[chave])
        ax.axvline(A.HW_THREADS, color=TINTA2, lw=1, ls=":")
        ax.text(A.HW_THREADS * 1.08, 0.97, "32 hw threads", transform=ax.get_xaxis_transform(),
                fontsize=6, color=TINTA2, va="top")
        ax.set_xscale("log", base=2)
        ax.set_xticks([1, 3, 7, 15, 31, 63, 127, 255, 511, 1023])
        ax.set_xticklabels(["1", "3", "7", "15", "31", "63", "127", "255", "511", "1023"],
                           fontsize=6, rotation=45)
        ax.minorticks_off()
        ax.set_ylim(bottom=0)
        ax.set_title(rotulo_n(n), fontsize=8, color=TINTA)
        ax.set_xlabel("processos (p)")
    eixos[0][0].set_ylabel("speed-up")
    alcas, nomes = eixos[0][0].get_legend_handles_labels()
    fig.legend(alcas, nomes, loc="lower center", ncol=2, fontsize=7, bbox_to_anchor=(0.5, -0.04))
    fig.suptitle("Excesso de processos: speed-up de 1 a 1023 processos", color=TINTA)
    fig.tight_layout(rect=(0, 0.05, 1, 1))
    salva(fig, d, "grafico_muitos")


def fig_ht(d, dados):
    ns = list(dados)
    rotulos, ganho_p, ganho_ht = [], [], []
    for n in ns:
        ht = A.carrega_ht(d, n)
        for v in ("arvore", "artigo"):
            t15, t31 = dados[n].get((v, 15)), dados[n].get((v, 31))
            th, tf = ht.get(("ht", v)), ht.get(("fisico", v))
            rotulos.append(f"{NOME[v]}\n{rotulo_n(n)}")
            ganho_p.append(t15["tempo"] / t31["tempo"] if t15 and t31 else 0)
            ganho_ht.append(tf / th if th and tf else 0)
    if not rotulos:
        return
    fig, ax = plt.subplots(figsize=(7.2, 2.6))
    x = range(len(rotulos))
    larg = 0.38
    ax.bar([i - larg / 2 - 0.01 for i in x], ganho_p, larg, color=COR["arvore"],
           label="15 →31 processos (árvore maior + HT)")
    ax.bar([i + larg / 2 + 0.01 for i in x], ganho_ht, larg, color=COR["artigo"],
           label="só HT: 31 em hw threads ×31 em núcleos físicos")
    ax.axhline(1, color=TINTA2, lw=1, ls=":")
    ax.set_xticks(list(x))
    ax.set_xticklabels(rotulos, fontsize=6)
    ax.grid(axis="x", visible=False)
    ax.set_ylabel("ganho (tempo antes / tempo depois)")
    ax.legend(loc="lower center", ncol=2, fontsize=7, bbox_to_anchor=(0.5, 1.0))
    ax.set_title("Ganho com hyper-threading", color=TINTA, pad=22)
    fig.tight_layout()
    salva(fig, d, "grafico_ht")


def main():
    d = sys.argv[1] if len(sys.argv) > 1 else "resultados"
    dados = A.carrega(d)
    if not dados:
        sys.exit(f"nenhum tempos_N.csv em {d}/")
    fig_escala(d, dados)
    fig_carga(d, dados)
    fig_muitos(d, dados)
    fig_ht(d, dados)


if __name__ == "__main__":
    main()
```